

Marzia Tahsin Section 1

AI disclosure

I used Claude Sonnet 5 to help me understand and generate some Python code and to clarify parts of the project questions. I reviewed and understood all of the material myself, and I independently wrote the final explanations and responses.

Question 1: Heuristic design and admissibility [15 points]

For each of the three cost measures—**steps**, **Scrabble**, and **frequency**—describe the heuristic implemented in your program.

For each heuristic:

STEPS

1. Give a precise formula or pseudocode.

$h(\text{node}) = \text{hamming_distance}(\text{node.state}, \text{goal})$

2. Explain what unavoidable remaining cost it lower-bounds.

The minimum number of actions remaining is the lower bound for this specific heuristic. Each change changes only one letter position at a time during each action this happens

$\text{state}[i] + c + \text{state}[i+1:]$

So a state that has a different character from the goal in hamming_distance needs at least a hamming_distance actions.

3. Explain why it cannot overestimate the true optimal remaining cost.

Under the steps cost measure, every action costs exactly 1, so the true remaining cost is exactly actions needed $\times 1$, which is at least hamming_distance $\times 1 = h(\text{node})$. So h never overestimates.

SCRABBLE

1. Give a precise formula or pseudocode.

$h(\text{node}) = \text{sum over each position } i \text{ where } \text{node.state}[i] \neq \text{goal}[i] \text{ of } 1$

2. Explain what unavoidable remaining cost it lower-bounds.

Each different position needs at least one action to fix and under the Scrabble cost measure, every action costs at least 1 point, even the cheapest possible replacement letter. So a state that is different from the goal in hamming_distance positions needs a remaining cost of at least hamming_distance $\times 1$.

3. Explain why it cannot overestimate the true optimal remaining cost.

We do not know which letters will replace the remaining different positions. Each one could cost anywhere from 1 to 10. If we assume every remaining letter costs the cheapest amount, 1, our estimate will always be less than or equal to the real cost.

So, hamming_distance $\times 1 = h(\text{node})$ never overestimates the remaining cost.

FREQUENCY

1. Give a precise formula or pseudocode.

$h(\text{node}) = \text{hamming_distance}(\text{node.state}, \text{goal})$

2. Explain what unavoidable remaining cost it lower-bounds.

Each different position needs at least one change. Every change costs at least 1 because $1 + \text{rarity}(w)$ is never less than 1. So if a word differs from the goal in hamming_distance positions, the remaining cost is at least: $\text{hamming_distance} \times 1$

3. Explain why it cannot overestimate the true optimal remaining cost.

h assumes that each needed change costs only 1 and ignores the extra rarity cost. Since rarity can only increase the real cost, the true remaining cost is always at least $\text{hamming_distance} \times 1 = h(\text{node})$. Therefore, h never overestimates.

Question 2: Does the heuristic actually help? [10 points]

Run both **uniform-cost search** and **A*** on:

- WHY → ASK
- BEAR → DUCK

Run each problem using all three cost measures. Report:

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps					
WHY → ASK	Scrabble					
WHY → ASK	Frequency					
BEAR → DUCK	Steps					
BEAR → DUCK	Scrabble					
BEAR → DUCK	Frequency					

Calculate the percentage reduction as:

$$100 \times \frac{\text{UCS expansions} - \text{A* expansions}}{\text{UCS expansions}}$$

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY→ASK	Steps	5	773	5	16	97.93%
WHY→ASK	Scrabble	9	756	9	353	53.31%
WHY→ASK	Frequency	61.826	834	61.826	43	94.84%
BEAR→DUCK	Steps	4	691	4	6	99.13%
BEAR→DUCK	Scrabble	11	2952	11	1376	53.39%
BEAR→DUCK	Frequency	50.068	1196	50.068	6	99.50%

None

why -> ask, cost=steps

UCS:

773 paths have been expanded and 119 paths remain in the frontier

Solution cost: 5

A*:

16 paths have been expanded and 73 paths remain in the frontier

Solution cost: 5

why -> ask, cost=scrabble

UCS:

756 paths have been expanded and 161 paths remain in the frontier

Solution cost: 9

A*:

353 paths have been expanded and 470 paths remain in the frontier

Solution cost: 9

why -> ask, cost=frequency

UCS:

834 paths have been expanded and 73 paths remain in the frontier

Solution cost: 61.82606767113925

A*:

43 paths have been expanded and 195 paths remain in the frontier

Solution cost: 61.82606767113925

bear -> duck, cost=steps

UCS:

691 paths have been expanded and 1409 paths remain in the frontier

Solution cost: 4

A*:

6 paths have been expanded and 54 paths remain in the frontier
Solution cost: 4

bear -> duck, cost=scrabble

UCS:

2952 paths have been expanded and 535 paths remain in the frontier

Solution cost: 11

A*:

1376 paths have been expanded and 1552 paths remain in the frontier

Solution cost: 11

bear -> duck, cost=frequency

UCS:

1196 paths have been expanded and 1380 paths remain in the frontier

Solution cost: 50.06764822972462

A*:

6 paths have been expanded and 54 paths remain in the frontier

A negative result means that A* expanded more nodes than UCS.

Then answer:

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.

Yes, both UCS and A* gave the same optimal cost in all six cases. This is normal because our heuristics are admissible, meaning they never overestimate the true remaining cost. When that holds, A* is guaranteed to find the same optimal solution as UCS, just with less searching.

2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?

why=>ask	Steps	$(100 \times (773-16))/773 = 97.93$
why=>ask	Scrabble	$(100 \times (756-353))/756 = 53.307$
why=>ask	Frequency	$(100 \times (834-43))/834 = 94.844$
bear=>duck	Steps	$(100 \times (691-6))/691 = 99.132$
bear=>duck	Scrabble	$(100 \times (2952-1376))/2952 = 53.388$
bear=>duck	Frequency	$(100 \times (1196-6))/1196 = 99.498$

The why=>ask pair had the largest percentage reduction of 97.93%

The bear=>duck pair had the largest percentage reduction of 99.498%

3. For each word pair, under which cost measure was the heuristic least informative?

Scrabble was the least informative cost measure for both pairs; 53.31% reduction for why=>ask and 53.39% for bear=>duck, roughly half the reduction seen under the other two measures.

4. Explain the differences using the definitions of your heuristics, not runtime alone.

The steps heuristic is strongest because every move costs exactly 1, so it closely estimates the remaining cost.

The Scrabble heuristic assumes each change costs 1, even though letters may cost 2–10. This makes its estimate less accurate, so A* rules out fewer paths.

The frequency heuristic also assumes a cost of 1 per move, but rare words can make a path's actual cost very high. A* avoids those expensive paths because of their high g-cost, not because the heuristic is very precise.

Different optimal paths are acceptable when their total costs are equal.

Question 3: Reversibility and direction-dependent costs [5 points]

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal.

For each cost measure—**steps**, **Scrabble**, and **frequency**—determine whether the optimal cost from W_1 to W_2 must always equal the optimal cost from W_2 to W_1 .

For each cost measure:

STEPS

1. State **yes** or **no**.

YES

2. Justify your conclusion.

With the steps cost measure, the cost is the same in both directions because every move costs 1. Also, if you can change w_1 to w_2 , you can reverse that move and change w_2 back to w_1 . Therefore, the shortest path from w_1 to w_2 has the same number of steps as the shortest path from w_2 to w_1 .

3. If the answer is no, give a concrete counterexample using words from the provided dictionary. Clearly distinguish between the reversibility of an action and the symmetry of its cost.

N/A

SCRABBLE

1. State **yes** or **no**.

NO

2. Justify your conclusion.

With Scrabble costs, a move is priced by the letter you add. Reversing the move adds a different letter, so it may have a different cost. The move is legal in both directions, but the cost does not have to be the same.

3. If the answer is no, give a concrete counterexample using words from the provided dictionary. Clearly distinguish between the reversibility of an action and the symmetry of its cost.

Counterexample: $\text{cat} \Rightarrow \text{cap}$ changes t to p , so it costs $\text{value}(p) = 3$. The reverse move, $\text{cap} \Rightarrow \text{cat}$, changes p to t , so it costs $\text{value}(t) = 1$.

Both words differ by only one letter, so one move is the cheapest path in each direction. Therefore, $\text{optimal}(\text{cat} \Rightarrow \text{cap}) = 3$, but $\text{optimal}(\text{cap} \Rightarrow \text{cat}) = 1$. The moves are reversible, but their costs are different.

FREQUENCY

1. State **yes** or **no**.

NO

2. Justify your conclusion.

With frequency cost, the cost depends on the word you move to: $1 + \text{rarity}(w)$. Reversing the move leads to a different word, which may have a different rarity value. So the move can be legal in both directions but have different costs.

3. If the answer is no, give a concrete counterexample using words from the provided dictionary. Clearly distinguish between the reversibility of an action and the symmetry of its cost.

Counterexample

Using the actual dictionary values, cat has frequency cost 12.8264 and cap has frequency cost 13.0127 (rarer words have higher cost).

Then: $\text{cost}(\text{cat} \Rightarrow \text{cap}) = \text{frequency_cost}(\text{cap}) = 13.0127$,
 $\text{cost}(\text{cap} \Rightarrow \text{cat}) = \text{frequency_cost}(\text{cat}) = 12.8264$.

Both moves are legal, but their costs differ because the destination words have different rarity values.

Question 4: LLM heuristic audit [10 points]

Give a generative AI system the following prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Submit:

1. The model used, the exact prompt, and its proposed heuristic. A complete conversation transcript is not required.

Model used: **GPT-5.6 Terra Light**

Prompt used:

None

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Proposed Heuristic :

Let g be the goal word. For each position i , define

$$L_i = \min_{\substack{w \text{ in dictionary} \\ w[i]=g[i]}} (1 + \text{rarity}(w)).$$

Then use

$$h_R(n) = \sum_{\{i: n[i] \neq g[i]\}} L_i.$$

Why it is admissible: for every position where n differs from g , any solution must eventually change that position to the goal letter. The action that makes that change enters a dictionary word whose letter at position i equals $g[i]$, so that action costs at least L_i . Since one action changes only one position, these required actions are distinct, so their lower-bound costs can be added.

Also, because rarity is nonnegative, every $L_i \geq 1$. Therefore,

$$h_R(n) \geq \sum_{\{i: n[i] \neq g[i]\}} 1 = h_0(n).$$

It is strictly more informative whenever at least one currently mismatched position i has $L_i > 1$, meaning every dictionary word with the needed goal letter in that position has positive rarity.

One important caveat: strict improvement cannot be guaranteed for every possible dictionary. If all relevant rarity values are 0, then $L_i = 1$ and $h_R = h_0$. In fact, if a one-step neighbor of the goal has zero rarity, no admissible heuristic can be strictly larger than $h_0 = 1$ there, because the true remaining cost is exactly 1.

2. A precise formula or pseudocode for the proposed heuristic.

$$h_R(n) = \sum_{i: n[i] \neq g[i]} \min_{\substack{w \in D \\ w[i]=g[i]}} (1 + \text{rarity}(w))$$

3. Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample.

Yes, it is admissible.

For every letter that is different from the goal, we must eventually change that letter to the correct goal letter. Changing one letter requires one move, and that move has a cost of at least the smallest possible cost for entering a word with that correct letter.

Since one move can fix only one letter, we add the minimum costs for all wrong positions.

So the heuristic can never be more than the real cheapest path cost to the goal. So, it is admissible.

4. If the proposal is invalid, make the smallest repair you can or replace it with a safe alternative.

The proposal is valid, so it does not need a repair for admissibility.

The only limitation is that it may equal h_0 if all of the relevant minimum rarity values are 00. In that case, it is still admissible, just not strictly more informative.

A small safe alternative is:

$$h(n) = \max(h_R(n), h_0(n) + \text{rarity}(g))$$

For n not equal to g and $h(g) = 0$

This is safe because every path must eventually enter the goal word g , which costs $1 + \text{rarity}(g)$. It is strictly larger than h_0 whenever the goal has positive rarity.

5. Test the verified or repaired heuristic against h_0 on the two frequency-cost problems from Question 2.

Problem	h_0 expanded nodes	Rarity-aware h_R expanded nodes	Reduction
why → ask	43	43	0%
bear → duck	6	6	0%

The rarity-aware heuristic found the same optimal cost as h_0 and expanded the same number of nodes: 43 for why⇒ask and 6 for bear⇒duck (0% reduction). Although h_R is theoretically more informative, many words have similarly low entering costs, so restricting a change to a specific position rarely increases the estimate. It remains admissible, but showed no practical benefit on this dictionary.